

Blokk 1

Sources og seeds

Hva er en source i dbt?

En **source** er en tabell som dbt ikke eier — den er lastet inn av et annet system.

Ved å registrere kilder i dbt får du:

- Mulighet til å referere til dem med `{{ source() }}` i stedet for hardkodede tabellnavn
- Automatisk linjeavstamning fra kilde til mart
- Freshness-sjekker som varsler når data er for gammelt

```
# models/sources.yml (filnavn. Et hvilket som helst gyldig finavn er ok)
sources:
  - name: crm # <- Navn på kilden
    schema: raw # <- Schema for kilden. Valgfritt; hvis ikke oppgitt brukes name
  tables:
    - name: crm_customers # <- navnet på tabellen i kilden
    - name: crm_contracts
```

`{{ source() }}` i en modell

I stedet for å skrive `raw.crm_customers` direkte i SQL bruker du:

```
select *  
from {{ source('crm', 'customers') }}
```

dbt peker til riktig database og schema basert på YAML-konfigurasjonen, slik at hvis schema-navnet endrer seg, endrer du det ett sted.

Freshness

dbt kan sjekke hvor ferske kildedataene er ved å lese en tidsserie-kolonne.

```
sources:
  - name: crm
    schema: raw
    loaded_at_field: _loaded_at # <- Hvilken kolonne som brukes til å angi rekkefølge
    freshness:
      warn_after: {count: 24, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: crm_customers
      - name: crm_contracts
      freshness:
        warn_after: {count: 6, period: hour} # override per tabell
```

Kjøres med:

```
dbt source freshness
```

Freshness — hva betyr terskelverdiene?

`warn_after` og `error_after` definerer hvor gammelt det nyeste innlastede datapunktet kan være.

- **warn**: dbt rapporterer en advarsel, jobben fortsetter
- **error**: dbt feiler — nyttig i en pipeline der downstream-kjøring ikke gir mening på foreldet data

Til diskusjon

- Hva er et realistisk freshness-SLA for hvert kildesystem?
- Bør CRM og billing ha samme terskel?
- Hva skjer i helger og helligdager?

Hva er et seed?

Et **seed** er en CSV-fil som dbt laster inn som en tabell i databasen.

Passer for små, stabile referansedata som ikke finnes i noe kildesystem — for eksempel en mapping-tabell eller en produktkatalog.

```
dbt-project/  
└─ seeds/  
   └─ catalog_products.csv
```

Lastes inn med:

```
dbt seed
```

Deretter kan andre modeller referere til den med `{{ ref('catalog_products') }}`.

Seeds i YAML

Seeds kan beskrives og konfigureres i YAML på samme måte som modeller.

```
# seeds/_seeds.yml
seeds:
  - name: catalog_products
    description: >
      Produktkatalog med alle abonnementstyper selskapet tilbyr.
      Inneholder historiske produktversjoner med valid_from / valid_to.
    columns:
      - name: product_code
        description: Unik produktkode. Kan gjenbrukes på tvers av versjoner.
      - name: valid_to
        description: Null for gjeldende produkter. Satt for utgåtte versjoner.
```

Du kan også overstyre kolonnetyper direkte i YAML:

```
config:
  column_types:
    monthly_fee_eur: float64
    valid_from: date
```


Kilder

Vi har data fra fire kildesystemer, alle i BigQuery-datasettet `raw`:

SYSTEM	TABELLER
CRM	<code>crm_customers</code> , <code>crm_contracts</code>
Fakturering	<code>billing_invoices</code> , <code>billing_payments</code>
Nettverk	<code>network_service_activations</code> , <code>network_incidents</code>
Produktkatalog	<code>catalog_products</code>

I dette blokket registrerer vi kildene og gjør oss kjent med dataene.

Oppgaver — I

1. Utforsk kildedataene

Åpne BigQuery og ta en titt på tabellene i `raw`-datasettet.

Legg merke til kolonnenavn, datatyper og et par rader med data.

Spørsmål å tenke på:

- Hvilke kolonner ser rare ut?
- Hva mangler?

2. Registrer kildene i YAML

Opprett filen `models/sources.yml` og registrer alle fire kildesystemer.

Krav:

- Bruk meningsfulle `name`-verdier for hver source-gruppe
- Sett `schema: raw` på alle
- Inkluder alle 7 tabeller

Sjekk at dbt leser konfigurasjonen uten feil:

```
dbt parse
```

3. Legg til freshness på CRM-kildene

`crm_customers` og `crm_contracts` har kolonnen `created_at` .

- Sett `loaded_at_field: created_at` på CRM-sourcen
- Definer fornuftige `warn_after` og `error_after` -verdier
- Kjør `dbt source freshness` og se hva som skjer

Hva rapporterer dbt? Er resultatet forventet gitt datoene i dataene?

Oppgaver II

4. Legg til beskrivelser

Legg til en `description:` på hver kildetabell i YAML.

Hold det kort — én setning som forklarer hva tabellen inneholder og hvilket system den kommer fra.

Diskusjonsspørsmål: Hvor bør man legge dokumentasjon?

5. Freshness per tabell

Ulike tabeller oppdateres med ulik frekvens.

Definer individuelle freshness-terskler per tabell i stedet for én felles per source.

Tenk igjennom:

- `billing_invoices` vs. `catalog_products` — bør de ha samme terskel?
- Hva er konsekvensen av en for streng terskel på en tabell som bare oppdateres månedlig?

6. meta-felter

dbt lar deg legge til vilkårlige nøkkelverdi-par under `meta:` på en source eller tabell.

```
tables:  
  - name: crm_customers  
    meta:  
      owner: crm-team  
      update_frequency: daily  
      pii: true
```

Legg til relevante `meta`-felter på minst to av kildene.

Disse er ikke funksjonelle i seg selv, men dukker opp i docs-siden og kan leses av verktøy.

Blokk 2

Staging-modeller og materialisering

Hva er en staging-modell?

En staging-modell er en **1-til-1-representasjon av én kildetabell** — rensset, men ikke tolket.

Det er det eneste stedet i prosjektet der vi:

- Gir kolonner konsistente navn
- Caster datatyper til riktige typer
- Løser opp kildesystemets tekniske særegenheter

Staging-modeller inneholder **ingen forretningslogikk** og **ingen joins**.

```
stg_crm__customers.sql    ← én modell per kildetabell
stg_crm__contracts.sql
stg_billing__invoices.sql
stg_billing__payments.sql
```

Hva hører hjemme i staging?

HØRER HJEMME	HØRER IKKE HJEMME
Kolonnenavn → standard navn	Joins mellom tabeller
<code>VARCHAR</code> → <code>DATE</code> , <code>NUMERIC</code>	Beregninger på tvers av rader
JSON-utpakking (enkle, flate felt)	Forretningsregler og flagg
Filtrering av tekniske metadata-rader	Aggregering
<code>COALESCE</code> for enkle standardverdier	Historisering / SCD-logikk

Navngiving i staging

Standardiser kolonnenavn på tvers av alle staging-modeller:

```
select
  -- rename: fjern kildesystemets særegenheter
  custID           as customer_id,
  full_name        as customer_name,

  -- cast: gi kolonnen riktig type
  cast(created_at as date)           as created_date,
  safe_cast(monthly_fee as numeric)  as monthly_fee_eur,
  parse_date('%Y%m%d', invoice_date) as invoice_date,

  -- beholdes som de er
  email,
  phone,
  status

from {{ source('crm', 'crm_customers') }}
```

`safe_cast` i BigQuery returnerer `null` i stedet for å feile ved ugyldige verdier — nyttig under utforsking.

Hva er materialisering?

Materialisering bestemmer **hvordan dbt skriver modellen til databasen**.

TYPE	HVA DBT GJØR	TYPISK BRUK
<code>view</code>	Oppretter en view — ingen data lagres	Staging, enkle transformasjoner
<code>table</code>	Materialiserer hele resultatet som en tabell	Tunge intermediate-modeller
<code>incremental</code>	Legger til / oppdaterer kun nye rader	Store faktatabeller, events
<code>ephemeral</code>	Kompileres inn som en CTE — finnes ikke i DB	Gjenbrukbare mellomsteg

Staging-modeller er nesten alltid `view` — de er tynne og brukes som grunnlag for neste lag, ikke som sluttpunkt.

Materialisering i YAML-konfigurasjon

Materialisering settes i YAML — ikke i SQL-filen.

```
# models/staging/_staging.yml
models:
  - name: stg_crm__customers
    config:
      materialized: view
```

Eller på mappenivå i `dbt_project.yml`:

```
models:
  mitt_prosjekt:
    staging:
      +materialized: view      # gjelder alle modeller i staging/
    intermediate:
      +materialized: table
    marts:
      +materialized: table
```

+ foran en nøkkel betyr at den arves nedover i mappestrukturen.

View vs. table

En `view` beregnes på nytt hver gang den spørres. Det er greit når:

- Modellen er rask å kjøre
- Den brukes sjelden eller av andre modeller, ikke sluttbrukere
- Du vil alltid se ferske data

Bytt til `table` når:

- Modellen tar lang tid å kjøre og refereres ofte
- Downstream-modeller bygger på den og du vil ikke gjøre jobben om igjen
- Sluttbrukere spør direkte mot modellen

Staging er nesten alltid `view`. Intermediate og mart er ofte `table`. Start med `view` og bytt når du har grunn til det.

Datasett: staging

Vi skal bygge én staging-modell per kildetabell:

STAGING-MODELL	KILDE	HVA SOM MÅ FIKSES
<code>stg_crm__customers</code>	<code>crm_customers</code>	<code>custID</code> → rename, JSON-felt, <code>created_at</code> cast
<code>stg_crm__contracts</code>	<code>crm_contracts</code>	<code>monthly_fee</code> VARCHAR → NUMERIC
<code>stg_billing__invoices</code>	<code>billing_invoices</code>	<code>InvoiceID</code> → rename, YYYYMMDD → DATE, <code>amount_eur</code> → NUMERIC
<code>stg_billing__payments</code>	<code>billing_payments</code>	Ryddig — minimalt å gjøre
<code>stg_network__service_activations</code>	<code>network_service_activations</code>	JSON <code>device_info</code> — diskusjon: her eller intermediate?
<code>stg_network__incidents</code>	<code>network_incidents</code>	<code>customer_ref</code> — behold navn eller rename?
<code>stg_catalog__products</code>	<code>catalog_products</code>	Ryddig — seed, bruker <code>ref()</code> ikke <code>source()</code>

Oppgaver — III

1. Bygg `stg_crm__customers`

Opprett `models/staging/stg_crm__customers.sql`.

Krav:

- Rename `custID` → `customer_id`
- Cast `created_at` til `DATE`
- Behold `address` og `preferences` som strenger (JSON pakkes ut senere)
- Referer til kilden med `{{ source('crm', 'crm_customers') }}`

Registrer modellen i `models/staging/_staging.yml` med `materialized: view`.

```
dbt run --select stg_crm__customers
```


2. Bygg `stg_billing_invoices`

`billing_invoices` har tre kolonner som trenger behandling.

Krav:

- Rename `InvoiceID` → `invoice_id`
- Cast `invoice_date` og `due_date` fra `YYYYMMDD`-streng til `DATE`
- Cast `amount_eur` fra `STRING` til `NUMERIC`

Tips for datokonvertering i BigQuery:

```
parse_date('%Y%m%d', invoice_date) as invoice_date
```

Hva skjer med de krediterte radene der `amount_eur` er negativ etter casten? Er det et problem her, eller hører det hjemme i en test?

3. Registrer alle staging-modeller i YAML

Opprett eller utvid `models/staging/_staging.yml` med alle 7 staging-modeller.

Krav:

- `materialized: view` på alle
- Minst én `description:` per modell
- Kjør alle modellene og verifiser at de kompilerer:

```
dbt run --select staging
```

Oppgaver — IV

4. Bygg de resterende staging-modellene

Bygg `stg_crm__contracts`, `stg_billing__payments`, `stg_network__incidents` og `stg_catalog__products`.

Merk:

- `stg_catalog__products` skal bruke `{{ ref('catalog_products') }}`, ikke `{{ source() }}` — hvorfor?
- `network_incidents` bruker `customer_ref` som fremmednøkkel. Skal du rename den til `customer_id`?

5. Materialisering på mappenivå

Flytt materialiseringskonfigurasjonen ut av enkeltmodellenes YAML og inn i `dbt_project.yml` som en mappenivå-setting.

```
models:
  mitt_prosjekt:
    staging:
      +materialized: view
```

- Fjern `materialized:` fra hver enkelt modell i `_staging.yml`
- Kjør `dbt run --select staging` og bekreft at ingenting endret seg

Hva er fordelen med å sette materialisering sentralt fremfor per modell? Når vil du overstyre det per modell?

6. JSON i staging

`crm_customers` har to JSON-kolonner: `address` (flat adressestruktur) og `preferences` (kommunikasjonspreferanser).

Pakk ut `address`-feltet i `stg_crm__customers`:

```
json_value(address, '$.street') as address_street,  
json_value(address, '$.city')   as address_city,  
json_value(address, '$.zip')    as address_zip,  
json_value(address, '$.country') as address_country,
```

Diskuter deretter:

- Er det riktig å pakke ut `address` i staging? Hva med `preferences`?
- Hva er argumentet for å vente til intermediate?
- Hva vil du gjøre med rader der `preferences` er `{}`?

Blokk 3

Intermediate-modeller

Hva er et intermediate-lag?

Intermediate-modeller kombinerer og beriker staging-modeller. De forbereder entiteter for konsumpsjon — uten å ta stilling til hvem som konsumerer dem.

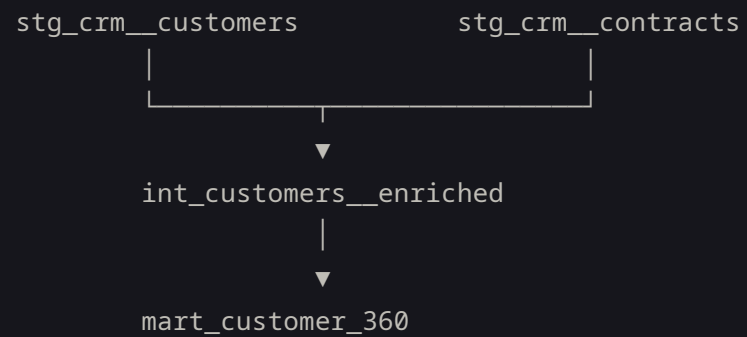
Typisk innhold:

- Joins mellom staging-modeller fra ulike kildesystemer
- Utpakking av JSON-felt som er for komplekse for staging
- Deduplisering av rader fra kilden
- Tekniske avledninger som gjelder entiteten generelt (f.eks. er kontrakten aktiv?)

```
int_customers__enriched.sql      ← CRM + kontrakter  
int_customers__activations.sql  
int_invoices__with_payments.sql
```

Navnekonvensjonen `int_<entitet>__<beskrivelse>` grupperer modeller etter entitet, ikke etter kilde.

Staging vs. intermediate vs. mart



En **staging**-modell: hva finnes i denne kildetabellen?

En **intermediate**-modell: hva vet vi om denne entiteten — uavhengig av hvem som spør?

En **mart**-modell: hva trenger dette spesifikke bruksområdet?

Hva skiller intermediate fra mart?

Intermediate inneholder logikk som er **universell for entiteten**:

- En kunde har én aktiv kontrakt (ikke konsumentspesifikt — det er alltid sant)
- En betaling er en duplikat hvis samme faktura er betalt to ganger (alltid sant)
- Kontraktsvarighet i dager (en egenskap ved kontrakten, ikke ved rapporten)

Mart inneholder logikk som er **spesifikk for konsumenten**:

- Livstidsverdi i euro (KPI for churn-teamet)
- Antall hendelser siste 90 dager (metrikk for nettverksdashboard)
- `is_at_risk` -flagg basert på betalingshistorikk (forretningsregel for én rapport)

*Det vanlige rådet er at «forretningslogikk hører hjemme i mart». Det stemmer for **konsumentspesifikk** logikk. Intermediate håndterer logikk som ville blitt duplisert på tvers av mange marts hvis den ikke ble løftet opp.*

JSON-utpakking — staging eller intermediate?

Dette er et designvalg, ikke et fasitsvar. To hensyn styrer beslutningen:

Pakk ut i staging når:

- JSON-feltet er flatt og enkelt
- Feltene brukes direkte som kolonner uten videre transformasjon
- Utpakkingen ikke avhenger av andre modeller
- Eksempel: `address` i `crm_customers`

Pakk ut i intermediate når:

- JSON-feltet er komplekst eller delvis null
- De utpakkede feltene må kombineres med data fra andre modeller for å gi mening
- Utpakket innhold er bare relevant i en spesifikk kontekst
- Eksempel: `preferences` og `device_info` — meningsfull først når vi vet hvem kunden er

Staging handler om kilden. Intermediate handler om entiteten.

JSON-utpakking i BigQuery

```
-- Flat utpakking med json_value()
json_value(preferences, '$.language')      as preferred_language,
json_value(preferences, '$.paperless')     as paperless_billing,
json_value(preferences, '$.marketing_opt_in') as marketing_opt_in,

-- Nested – device_info fra network_service_activations
json_value(device_info, '$.make')          as device_make,
json_value(device_info, '$.model')         as device_model,
json_value(device_info, '$.os')            as device_os,
json_value(device_info, '$.imei')          as device_imei,
```

`json_value()` returnerer alltid `STRING`. Cast eksplisitt hvis du trenger en annen type:

```
cast(json_value(preferences, '$.paperless') as bool) as paperless_billing
```

Tomme JSON-objekter `{}` gir `null` på alle `json_value()`-kall — håndter dette med `COALESCE` eller en test.

Tags i YAML

Tags lar deg gruppere modeller på tvers av mapper — nyttig for selektiv kjøring og dokumentasjon.

```
models:
  - name: int_customers__enriched
    config:
      materialized: table
      tags:
        - customer
        - daily
    columns:
      - name: customer_id
        description: Unik kundeidentifikator. Primærnøkkel.
      - name: preferred_language
        description: Foretrukket språk fra kundens preferanseprofil. Null hvis ikke satt.
```

Bruk tags til å kjøre et utvalg:

```
dbt run --select tag:customer
dbt build --select tag:daily
```

Kolonnenivå-beskrivelser i YAML

Beskrivelser på kolonnenivå fyller dokumentasjonssiden med innhold.

```
models:
  - name: int_customers__enriched
    description: >
      Én rad per kunde, beriket med aktiv kontraktinformasjon og
      utpakkede preferanser fra CRM.
    columns:
      - name: customer_id
        description: Primærnøkkel. Arvet fra stg_crm__customers.
      - name: contract_status
        description: >
          Status på kundens siste aktive kontrakt.
          Null for kunder uten noen kontrakt (f.eks. status pending).
      - name: monthly_fee_eur
        description: Månedlig totalbeløp på tvers av alle aktive kontrakter.
```

Skriv beskrivelsen slik at en ny kollega forstår kolonnen uten å se SQL-en.

Deduplisering

`billing_payments` inneholder dupliserte betalingsrader — samme faktura betalt to ganger i systemet.

En vanlig tilnærming med `ROW_NUMBER()`:

```
with payments_deduped as (  
  select  
    *,  
    row_number() over (  
      partition by invoice_ref  
      order by paid_on  
    ) as rn  
  from {{ ref('stg_billing__payments') }}  
)  
  
select * from payments_deduped  
where rn = 1
```

Spørsmål å tenke på:

- Hvilken rad beholder du — den første eller siste?
- Bør du filtrere i intermediate, eller eksponere `rn` til mart-laget?
- Burde det også finnes en **test** som fanger dette problemet?

Datasett: intermediate

Modellene vi skal bygge i denne blokken:

INTERMEDIATE-MODELL	HVA DEN GJØR
<code>int_customers__enriched</code>	Kobler <code>stg_crm__customers</code> med siste aktive kontrakt, pakker ut <code>preferences</code>
<code>int_customers__activations</code>	Kobler kunder med siste nettverksaktivering og pakker ut <code>device_info</code>
<code>int_billing__invoice_settlement</code>	Fakturaer beriket med betalingsstatus — faktura er primærentiteten, betaling er et oppslag
<code>int_network__incidents_enriched</code>	Kobler hendelser med kundeinformasjon

`crm_contracts` og `catalog_products` inneholder allerede historikk med `valid_from` / `valid_to` fra kildesystemet — staging eksponerer det som det er.

Oppgaver — V

1. Bygg `int_customers__enriched`

Opprett `models/intermediate/int_customers__enriched.sql`.

Krav:

- Join `stg_crm__customers` med `stg_crm__contracts`
- Behold kun **siste aktive kontrakt** per kunde (hint: `ROW_NUMBER()` over `start_date`)
- Pakk ut `preferences` -JSON: `preferred_language` , `paperless_billing` , `marketing_opt_in`
- Håndter kunder uten noen kontrakt (C017 har ingen)

Registrer modellen i `models/intermediate/_intermediate.yml` med `materialized: table` og minst tre kolonnenivå-beskrivelser.

```
dbt run --select int_customers__enriched
```

2. Bygg `int_billing__invoice_settlement`

Her møter vi en reell designutfordring: faktura og betaling er to separate entiteter — men vi skal likevel koble dem i intermediate. Er det i tråd med prinsippet om at intermediate handler om én entitet?

To gyldige tilnærminger:

Tilnærming A — faktura som primærentitet (det vi gjør her):

Modellen har én rad per faktura. Betalingen er ikke en likeverdig entitet — den er et oppslag som svarer på spørsmålet *er denne fakturaen gjort opp?* Joinen tjener fakturaen. Modellnavnet `int_billing__invoice_settlement` signaliserer dette tydelig.

Tilnærming B — hold dem adskilt:

`int_billing__invoices` og `int_billing__payments_deduped` er separate modeller. Joinen skjer først i mart, der konsumenten faktisk trenger begge. Dette er renere i teorien, men betyr at dedupliseringslogikken for betalinger ikke er gjenbrukbar uten en egen modell for det.

Begge er forsvarbare. Valget avhenger av om deduplisering av betalinger er noe mange marts vil trenge — i så fall løfter du det opp. Hvis bare én mart noensinne bruker det, kan det like gjerne leve der.

Implementer tilnærming A:

Opprett `models/intermediate/int_billing__invoice_settlement.sql`.

Krav:

3. Legg til tags og kolonnenivå-beskrivelser

Utvid `_intermediate.yml` med:

- Tag `customer` på alle modeller som handler om kunder
- Tag `billing` på faktura- og betalingsmodellene
- Fullstendige kolonnenivå-beskrivelser på alle kolonner i `int_customers__enriched`

Verifiser at tags fungerer:

```
dbt run --select tag:customer
```

Oppgaver — VI

4. Bygg `int_customers__activations`

Koble `stg_crm__customers` med `stg_network__service_activations`.

Utfordringer å håndtere:

- Noen kunder har to aktiveringsrader (oppgradering av enhet) — beholder du den nyeste?
- `device_info` inneholder null-verdier for `imei` og `os` — håndter dette eksplisitt
- Pakk ut alle fire device-felter og cast `os` til lowercase for konsistens

5. Bygg `int_network__incidents_enriched`

Koble `stg_network__incidents` med `int_customers__enriched` for å berike hendelsene med kundeinformasjon.

Krav:

- Join på `customer_ref` → `customer_id` (merk: ulike kolonnenavn i de to modellene)
- Inkluder `customer_name`, `status` og `contract_status` fra kundemodellen
- Beregn `resolution_time_minutes` for løste hendelser (`resolved_at is not null`)
- Behold uløste hendelser i resultatet — ikke filtrer dem bort

Legg til modellen i `_intermediate.yml` med tag `network` og kolonnenivå-beskrivelser.

6. Intermediate som gjenbrukbart lag

`int_customers__enriched` er nå bygget. Se for deg at du skal lage to ulike marts:

- `mart_customer_360` — én rad per kunde med alle attributter
- `mart_churn_candidates` — kunder med suspendert status og åpne hendelser

Spørsmål:

- Hvilken logikk ville blitt duplisert i begge marts hvis intermediate ikke fantes?
- Er det noe i `int_customers__enriched` som egentlig er konsumentspesifikt og burde vært i mart i stedet?
- Hvor går grensen i akkurat dette tilfellet?